PROGRAMMING & CS FUNDAMENTALS

Big O Notation Visual Guide

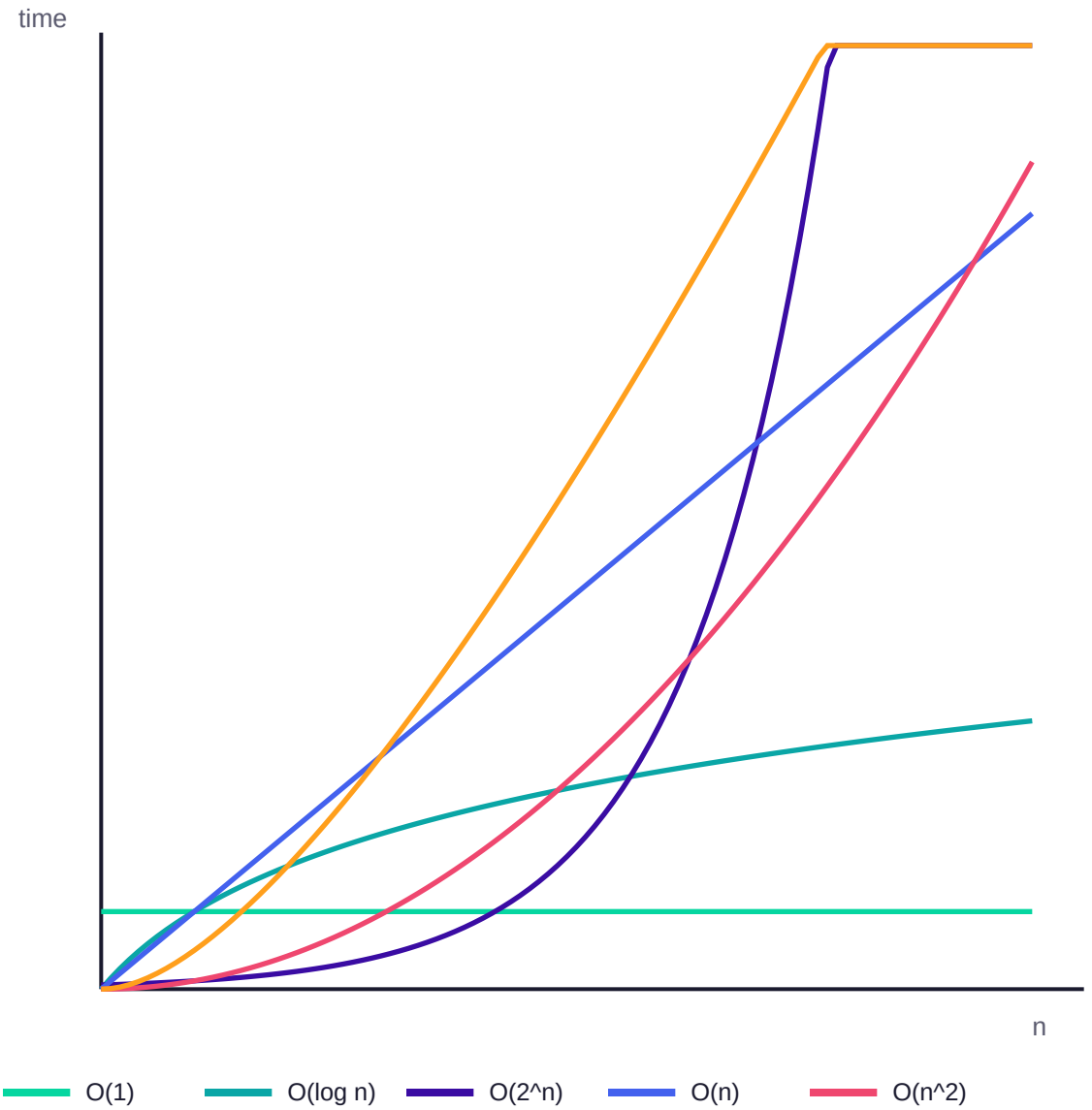
How runtime grows with input size —
the one idea behind every performance claim.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Growth Curves: The Complexity Zoo

Shape matters more than constants: compare how each class explodes as n grows.



Same n, wildly different bills: at n=1M, O(n) is a million ops, O(n^2) is a trillion. Pick the shape first, optimize constants later.

STEP 2/6

Reading Big O: The Three Rules

Simplify any expression down to its dominant shape.

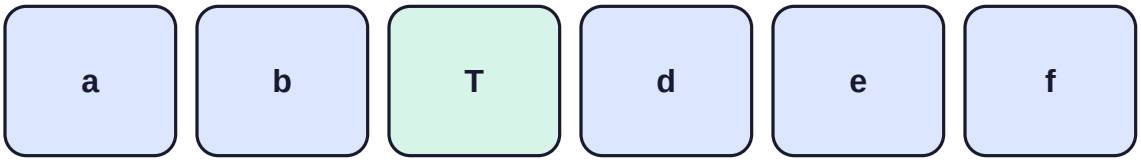
```
3n^2 + 5n + 100      ->  O(n^2)     # drop lower-order terms
2n                   ->  O(n)       # drop constants
O(n) + O(log n)      ->  O(n)       # keep the dominant term
nested loops n x n   ->  O(n^2)
loop halves n        ->  O(log n)   # binary search
recursion 2T(n/2)+n  ->  O(n log n) # merge sort
```

- Rule 1 — drop constants: O(2n) is O(n). Hardware speed is not the algorithm's business.
- Rule 2 — drop lower-order terms: for large n, n^2 dwarfs 1000n.
- Rule 3 — worst case by default: unless stated, Big O describes the worst-case input.
- Log bases don't matter: log2 n and log10 n differ by a constant factor.

STEP 3/6

Best / Average / Worst Case

One algorithm, three stories — linear search for target T.



Best case: T is first -> O(1). One comparison and done.

Average case: T is anywhere -> O(n/2) = O(n).

Worst case: T is last or absent -> O(n). Every element checked.

```
for x in arr:            # linear search
    if x == T: return i   # best: 1 step, worst: n steps
```

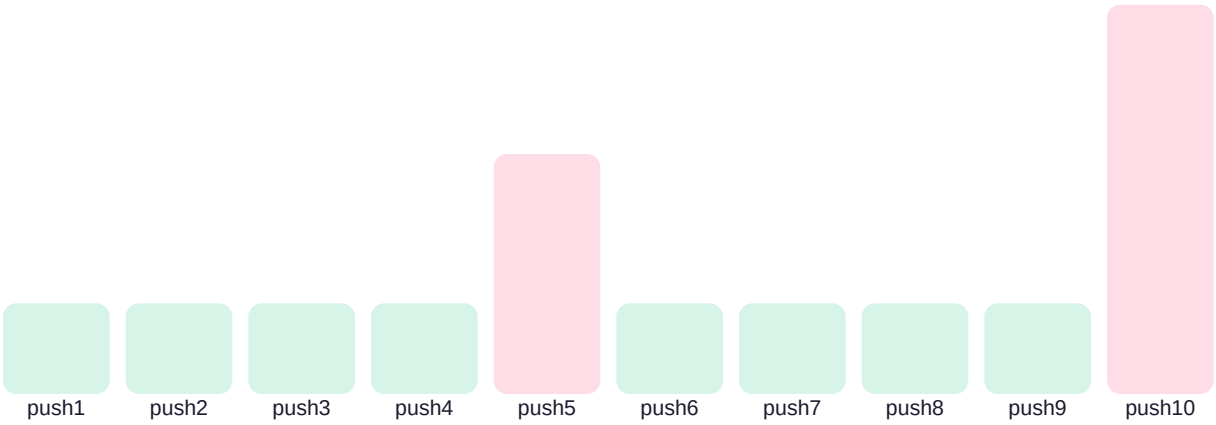
Interviews test worst case; production cares about average case. Know which one you're quoting — and say it out loud.

STEP 4/6

Amortized Analysis

Occasional expensive ops, averaged over a long sequence.

Appending to a dynamic array (capacity doubles on fill):



Most pushes cost 1 (red bars = resize + copy). Total for n pushes < 3n -> O(1) amortized.

- Aggregate method: bound the total cost of n operations, divide by n.
- Amortized O(1) is not worst-case O(1): a single push can still cost O(n).
- Real-time systems hate amortization spikes — they need true worst-case bounds.

STEP 5/6

Space Complexity

Time gets the glory; memory decides whether it runs at all.

O(1) extra

in-place swap, two pointers

O(log n)

balanced recursion depth

O(n)

copy of input, hash map of n keys

O(n^2)

adjacency matrix for n nodes

```
def reverse(arr):          # O(1) extra space
    i, j = 0, len(arr)-1  # two pointers, no copy
    while i < j: arr[i], arr[j] = arr[j], arr[i]; i+=1; j-=1
```

Count auxiliary space (beyond the input). Recursion depth counts — a 'clever' recursive O(n) can hide O(n) stack space.

STEP 6/6

Big-O Gotchas

Where smart people misread the notation.

- Small n: constants dominate. O(n^2) with tiny constants beats O(n log n) for n under 100.
- Hidden constants: O(n) with a 10-second constant loses to O(n^2) with a 1ms one — at small n.
- Big O is an upper bound. Tight bound = Big Theta; lower bound = Big Omega.
- 'O(n) time, O(1) space' is the gold standard for array problems — aim for it.
- Amortized != average != worst case. Name the analysis you're using.
- n must be defined: O(n*m) for an n x m grid, O(V+E) for graphs.

STEP ✓I✓

Key Takeaways

Big O Notation — Remember

- Growth shape beats constants: compare O(n) vs O(n^2) before micro-optimizing.
- Simplify with three rules: drop constants, drop lower-order terms, keep the dominant term.
- Quote best/average/worst explicitly — linear search is O(1), O(n), O(n) across the three.
- Amortized O(1) (dynamic array append) averages rare expensive resizes over many cheap ops.
- Space counts too: recursion depth and copies are real memory costs.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev